

Technical Report for the IMU Odometry Challenge

Cross-Platform Neural Inertial Odometry with Kinematically Consistent Re-Timing Augmentation

Kaggle team name: Haozhe Zhou

Haozhe Zhou¹

Abstract—Inertial Measurement Units (IMUs) provide odometry without external infrastructure or degradation under visual occlusion and darkness. The difficulty lies in the limited observability of absolute velocity from inertial measurements and integration errors, which learning-based approaches overcome by exploiting a motion prior. Yet such priors are intrinsically different across embodiments, such as between a quadruped and a drone, which makes cross-platform odometry especially challenging. This report presents our solution to the Tartan IMU Challenge (IROS 2026) aiming at a single model for inertial odometry across four platforms: wheeled, handheld, legged, and aerial. We use an encoder–decoder transformer that estimates the mean body-frame velocity of 1-second windows from raw 6-axis IMU inputs. To address the scarcity of data per platform, we re-time and re-scale the trajectories and IMU measurements while preserving kinematic consistency, leaving gravity and sensor bias unchanged. Our method achieves an overall score of 0.21367 over all 89 test sequences (AVE 0.17342 m/s, ATE₂₀ 0.56268 m, RTE 0.81807 m), a 60% reduction from the released TartanIMU baseline (0.53799). Ablation on the official test set shows that the re-timing augmentation alone improves the score by 22% from 0.27261.

I. INTRODUCTION

Inertial measurement units (IMUs) are a compact and low-cost modality for interoceptive perception. They have been widely used in pedestrian navigation [1]–[4], vehicle navigation [5], [6], robot odometry [7]–[10], and human motion capture [11], [12]. Unlike cameras or LiDAR, an IMU measures the motion of the body itself, so it requires no external infrastructure and is not affected by visual occlusion, darkness, or dynamic scenes. Its weakness is drift. Integrating acceleration and angular rate accumulates sensor noise and bias quadratically in time, and neither heading nor velocity is observable from inertial data alone [13]. Classical remedies constrain the integration with motion models or zero-velocity updates [1], [5]; recently, learning-based methods [3], [4], [10], [14]–[17] have demonstrated improved accuracy by utilizing a learned prior on the motion patterns, and learned components have also been used to denoise or debias the sensor inside a filter [6], [8], [18]. A car moves along its longitudinal axis; a quadruped’s gait frequency scales with its speed; a quadrotor balances aerodynamic drag with a tilt that appears in the specific force [19]; a human’s trajectory can largely be estimated through foot contacts and strides [20]. These regularities constrain the problem space, but they are intrinsically different across embodiments. As a result,

a model trained on one platform often does not transfer to another [9], [10], and collecting enough data to train a separate model for every platform is expensive.

Cross-platform inertial odometry has therefore become an active direction. TartanIMU [9] pretrains a shared trunk across platforms and attaches platform-specific heads, and shows that this outperforms models trained on each platform alone. However, it still requires the platform identity to select the head. The Tartan IMU Challenge at IROS 2026 goes one step further: it asks for a single model with one set of weights that predicts body-frame velocity for wheeled, legged, aerial, and handheld platforms, with no platform label at inference and no dispatch to per-platform experts. The model has to infer its embodiment from the signal itself, and it has to learn the motion priors of four platforms from only a few hours of data each.

We present our solution to this challenge. We use an encoder–decoder transformer that estimates body-frame velocity over 64 s of raw 6-axis IMU, with four learned time queries per 1-second window. We use a strided convolutional stem and a gated multi-resolution log-spectrogram branch to represent both transient and rhythmic features. The network also computes a continuous embodiment state vector from the input and uses it to modulate the normalization layers [21]. To address data scarcity, we re-time and re-scale trajectories and IMU inputs, with the corresponding changes to velocity, angular rate, and the kinematic part of the specific force, while gravity and sensor bias are left unchanged. The model is trained from scratch on the data from the challenge release only. Overall, our method achieves 0.21367 on the official scoring service over all 89 test sequences.

To summarize, our key contributions are:

- A single-weight cross-platform model that serves wheeled, legged, aerial, and handheld platforms without platform labels, classification, or routing.
- A simple yet effective IMU data augmentation technique that re-times and re-scales the data under kinematic consistency.

II. METHOD

A. Problem Definition

A 6-axis IMU rigidly attached to a moving platform provides, at 200 Hz, the specific force $\mathbf{a}_t$ and the angular rate $\boldsymbol{\omega}_t$ in the sensor frame; we write $\mathbf{x}_t = (\mathbf{a}_t, \boldsymbol{\omega}_t)$ for one sample. A recording is split into consecutive one-second

¹Haozhe Zhou is with Carnegie Mellon University, Pittsburgh, Pennsylvania, USA. haozhezh@cs.cmu.edu

windows $\mathcal{W}_k$, and the quantity to be estimated for window k is the mean velocity of the sensor in its own frame,

$$\hat{\mathbf{v}}_k = \frac{1}{|\mathcal{W}_k|} \sum_{t \in \mathcal{W}_k} \mathbf{R}_t^\top \dot{\mathbf{p}}_t, \quad (1)$$

where $\mathbf{R}_t \in SO(3)$ and $\mathbf{p}_t$ are the reference orientation and position of the sensor in the world frame. Our goal is to learn a single model f_θ that predicts $\hat{\mathbf{v}}_{1:K} = f_\theta(\mathbf{x}_{1:N})$ for every platform from the raw readings alone: the platform is not given, and the reference $\mathbf{R}_t, \mathbf{p}_t$ is used only to construct the targets (1) and the augmentations of Section II-C.

B. Overall Architecture

We realize f_θ as an encoder–decoder transformer [22] that processes a chunk of T consecutive windows at once, as shown in Fig. 1. The raw readings $\mathbf{x}_{1:N}$ are first turned into a token sequence $\mathbf{Z}$; the tokens are summarized into an embodiment state vector $\mathbf{c}$; the encoder maps $\mathbf{Z}$, conditioned on $\mathbf{c}$, to a memory $\mathbf{H}$; and the decoder reads the velocity $\hat{\mathbf{v}}_k$ of each window out of $\mathbf{H}$, again conditioned on $\mathbf{c}$. One set of weights serves all platforms.

Tokenizer. A strided convolutional stem compresses $\mathbf{x}_{1:N}$ into tokens at 8 Hz. Convolutions capture transient motion but represent rhythmic motion poorly, although the frequency of gait or vibration is what carries the speed of legged and wheeled platforms. We therefore compute, in parallel, a multi-resolution log-spectrogram of the same readings, project it to the token width, and add it to the convolutional tokens through a learned gate. The result is the token sequence $\mathbf{Z} \in \mathbb{R}^{S \times d}$.

Embodiment state vector. To describe the body that produced the chunk, we pool the tokens over time. Let $\boldsymbol{\mu}, \boldsymbol{\sigma}, \mathbf{m} \in \mathbb{R}^d$ denote the element-wise mean, standard deviation and maximum of the S tokens of $\mathbf{Z}$. A small MLP maps their concatenation to the embodiment state vector

$$\mathbf{c} = \text{MLP}_c([\boldsymbol{\mu}; \boldsymbol{\sigma}; \mathbf{m}]) \in \mathbb{R}^d. \quad (2)$$

These statistics capture the energy, spread and extremes of the motion in the chunk, which already separates a car from a quadruped from a drone; the MLP learns the finer distinctions. Every normalization layer ℓ of the encoder and the decoder is conditioned on $\mathbf{c}$ by scaling and shifting its normalized activations channel by channel, following FiLM [21] and the adaLN-Zero block of DiT [23],

$$\text{AdaLN}_\ell(\mathbf{h}; \mathbf{c}) = \text{LN}(\mathbf{h}) \odot (1 + \gamma_\ell(\mathbf{c})) + \beta_\ell(\mathbf{c}), \quad (3)$$

where the scale $\gamma_\ell(\mathbf{c})$ and the shift $\beta_\ell(\mathbf{c})$ are linear functions of $\mathbf{c}$ with a separate weight matrix $\mathbf{W}_\ell$ per layer. With $\mathbf{W}_\ell$ initialized to zero, the model starts as an ordinary transformer and learns during training how each layer should be adjusted for the chunk at hand. This is the only mechanism through which the model specializes to an embodiment. It is continuous and inferred from the signal, without classification or routing, so the same weights also adapt to different motion regimes within a platform.

Encoder. A stack of pre-norm transformer blocks [24] with bidirectional self-attention and rotary position embedding [25], each normalization adapts to the embodiment state by (3), maps $\mathbf{Z}$ to the memory $\mathbf{H} \in \mathbb{R}^{S \times d}$. A depthwise convolution in every block, as in the Conformer [26], gives each token a fixed local context of about one second in addition to the global attention.

Decoder. For each window k we place $Q = 4$ learned queries [27], one per quarter-second sub-window, initialized from a slot embedding plus a global summary of $\mathbf{H}$. The queries cross-attend to $\mathbf{H}$, self-attend to each other, and an MLP head maps each of them to a sub-window velocity; $\hat{\mathbf{v}}_k$ is the mean of the four. Predicting at sub-window granularity gives the model denser supervision than the one-second target (1) alone.

Auxiliary heads. During training, a second head predicts the direction of gravity in the sensor frame for each window, and linear classifiers on $\mathbf{c}$ and on the global summary predict the platform. Both serve only as auxiliary losses (Section II-E); they are not evaluated at inference and are removed from the released model. Layer counts and other hyper-parameters are listed in Section II-E.

C. Data Processing

We feed the model the raw 200 Hz readings without filtering, resampling, bias correction or frame canonicalization; the accelerometer is divided by $g = 9.81 \text{ m/s}^2$ so that both channels are of order one. A training sample is a chunk of $T = 64$ consecutive windows: we draw a platform uniformly, a trajectory within the platform with probability proportional to the square root of its length, and a random start time. Trajectories shorter than the chunk are zero-padded and masked. The targets are the mean body-frame velocities (1) of the four 0.25 s sub-windows of every window.

Kinematically consistent augmentation. Each platform contributes only a few hours of data, but conventional augmentation breaks the physics that ties the readings to the targets. Random rotation or mixing removes the motion regularities the model relies on, and even the naive time-warping of the raw signal used in time-series classification [28], [29] is kinematically inconsistent: it rescales gravity and bias together with the motion. We therefore transform the trajectory and derive the readings it would have produced. With the reference attitude $\mathbf{R}_t$, the specific force decomposes as

$$\mathbf{a}_t = \mathbf{a}_t^{\text{kin}} + g \mathbf{R}_t^\top \mathbf{e}_z + \mathbf{b}, \quad \mathbf{a}_t^{\text{kin}} = \mathbf{R}_t^\top \ddot{\mathbf{p}}_t, \quad (4)$$

where $\mathbf{a}_t^{\text{kin}}$ is the kinematic acceleration of the sensor, the second term is gravity, and $\mathbf{b}$ is a constant accelerometer bias, estimated per trajectory as the mean of $\mathbf{a}_t - \mathbf{R}_t^\top (\ddot{\mathbf{p}}_t + g \mathbf{e}_z)$ over the recording. The transformations act on $\mathbf{a}_t^{\text{kin}}$ only; gravity and bias do not depend on how the path is traversed.

Re-timing replays the trajectory along a time map $\tau(t)$ with a *smooth* random rate $k(t) = \dot{\tau} \in [0.5, 1.8]$, i.e. $\mathbf{p}'(t) = \mathbf{p}(\tau(t))$. By the chain rule, $\dot{\mathbf{p}}' = k \dot{\mathbf{p}}$ and $\ddot{\mathbf{p}}' = \frac{d}{dt}(k \dot{\mathbf{p}}) = k^2 \ddot{\mathbf{p}} + \dot{k} \dot{\mathbf{p}}$, so that in the sensor frame velocity, angular rate

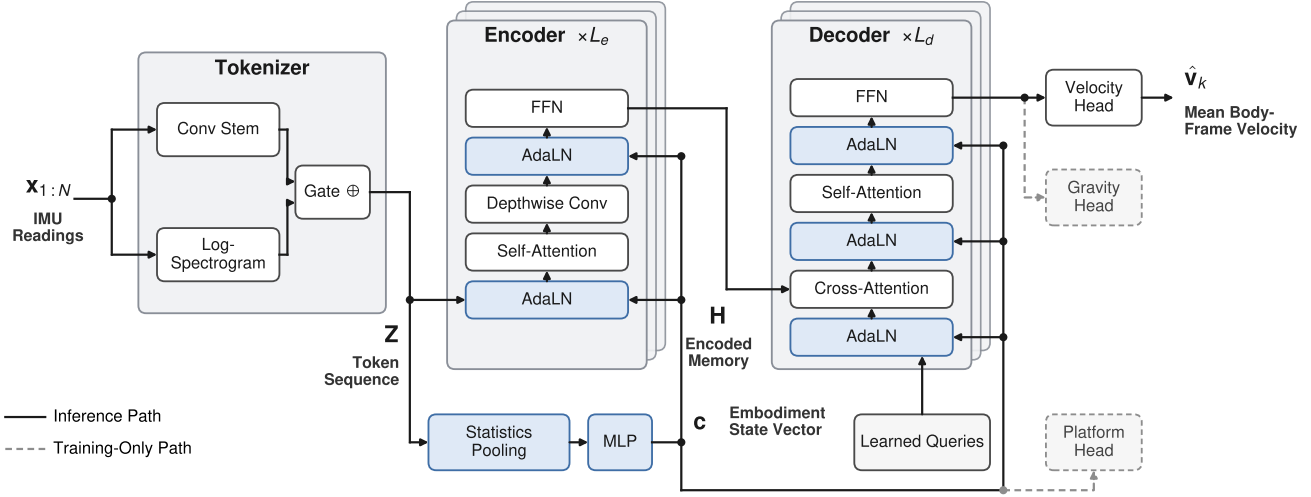


Fig. 1. Overview of the model. The raw readings $\mathbf{x}_{1:N}$ are tokenized into $\mathbf{Z}$, encoded into the memory $\mathbf{H}$, and decoded into one velocity $\hat{\mathbf{v}}_k$ per window. The embodiment state vector $\mathbf{c}$ (2) is pooled from the tokens and supplies the scale and shift of every AdaLN block (3). Grey dashed paths and boxes are used only for auxiliary losses during training.

and kinematic acceleration transform as

$$\mathbf{v}' = k \mathbf{v}, \quad \boldsymbol{\omega}' = k \boldsymbol{\omega}, \quad \mathbf{a}'^{\text{kin}} = k^2 \mathbf{a}^{\text{kin}} + \dot{k} \mathbf{v}, \quad (5)$$

with all quantities on the right evaluated at $\tau(t)$. The last term is the acceleration caused by the changing replay rate; naive time-warping omits it. *Re-scaling* traverses the same path scaled in space by $s \in [0.7, 1.4]$: $\mathbf{v}' = s \mathbf{v}$, $\mathbf{a}'^{\text{kin}} = s \mathbf{a}^{\text{kin}}$, $\boldsymbol{\omega}$ unchanged. The augmented readings are reassembled with (4), and the targets are recomputed from the transformed trajectory. Both transformations preserve the motion pattern of the platform: a re-timed car still moves along its longitudinal axis, and a re-timed quadruped still has a gait whose frequency matches its speed. Additive sensor noise, bias perturbation, and random mounting rotation did not improve our experiments and were not used.

D. Training Data

TABLE I
DATASETS USED FOR MODEL DEVELOPMENT AND TRAINING.

Dataset	Type	Usage
Challenge train	Official	Training data for all experiments
Challenge val	Official	Model selection; added to training of the final submitted model

We use only the released challenge data (Table I), expanded on the fly by the kinematically consistent augmentation of Sec. II-C. All development runs are trained on `train` with `val` held out for model selection; the submitted model is retrained with the same configuration on `train` and `val` together. No external data, synthetic data, pseudo-labels or pretrained weights are used, and the test IMU is never used for training or adaptation.

E. Training and Optimization

The model is trained from scratch in a single stage with five loss terms. The first is the velocity error that the challenge scores. The metric, however, also integrates the velocities into 20 m paths, so errors that are small per window but consistent in direction accumulate; we therefore also penalize the displacement error integrated with the reference attitude over multi-second spans. Third, since the decoder predicts four sub-window velocities behind every $\hat{\mathbf{v}}_k$, we supervise them directly with their 0.25 s targets $\bar{\mathbf{u}}_{k,j}$, which gives four times denser supervision than the window mean alone. The last two terms train the auxiliary heads: the gravity direction, which forces the trunk to separate tilt from motion, and the platform label, which shapes $\mathbf{c}$ into an embodiment descriptor. With $\hat{\mathbf{u}}_{k,j}$ and $\hat{\mathbf{g}}_k$ the sub-window and gravity outputs, the terms are

$$\begin{aligned} \mathcal{L}_{\text{vel}} &= \frac{1}{K} \sum_k \|\hat{\mathbf{v}}_k - \bar{\mathbf{v}}_k\|_2, \\ \mathcal{L}_{\text{disp}} &= \frac{1}{|\mathcal{H}|} \sum_{H \in \mathcal{H}} \frac{1}{H} \left\| \sum_{i=k}^{k+H-1} \mathbf{R}_i (\hat{\mathbf{v}}_i - \bar{\mathbf{v}}_i) \right\|_2, \\ \mathcal{L}_{\text{sub}} &= \frac{1}{KQ} \sum_{k,j} \ell_\delta(\hat{\mathbf{u}}_{k,j} - \bar{\mathbf{u}}_{k,j}), \\ \mathcal{L}_{\text{grav}} &= \frac{1}{K} \sum_k \|\hat{\mathbf{g}}_k - \mathbf{R}_k^\top \mathbf{e}_z\|_2, \\ \mathcal{L}_{\text{plat}} &= -\log \hat{p}(y | \mathbf{c}), \end{aligned} \quad (6)$$

with horizons $\mathcal{H} = \{5, 10, 20\}$ s, averaged over windows k ; ℓ_δ the smooth L_1 loss with $\delta = 0.1$; and $\hat{p}(y | \mathbf{c})$ the probability that a linear classifier on $\mathbf{c}$ assigns to the true platform y , i.e. a cross-entropy. The total loss is $\mathcal{L}_{\text{vel}} + 0.5 \mathcal{L}_{\text{disp}} + 0.5 \mathcal{L}_{\text{sub}} + \mathcal{L}_{\text{grav}} + 0.1 \mathcal{L}_{\text{plat}}$; the weights were set once and not tuned.

We use AdamW [30] with a learning rate of 5×10^{-4} , weight decay 0.05, 500 warm-up steps and cosine decay to zero, a batch of 32 chunks of 64 s, and fp16 mixed precision [31]. Development runs take 20k steps; the submitted model takes 30k steps, about 2.8 h on one NVIDIA Quadro RTX 8000. Validation curves are monotonic to the end of

the schedule, so the checkpoint is always the last step. The architecture uses $L_e = 6$ encoder and $L_d = 3$ decoder blocks of width $d = 256$ with 8 attention heads, a depthwise convolution of 7 tokens, and a spectral branch with four STFT windows of 0.25–2 s pooled into 24 log-spaced bands; the released model has 12.9M parameters.

F. Inference

A test trajectory is cut into 64 s chunks with a 32 s stride. Each chunk is one forward pass of the same model for every platform, and the overlapping predictions are blended with a squared triangular window; trajectories shorter than a chunk are processed in one pass. The submitted CSV is the network output: no ensembling, checkpoint averaging, test-time augmentation, smoothing or rescaling is applied.

G. Model Complexity and Runtime

TABLE II
MODEL COMPLEXITY AND RUNTIME STATISTICS.

Property	Value
Number of parameters	12.9M
Training hardware	1 × Quadro RTX 8000
Training time	2.8h
Inference hardware	1 × Quadro RTX 8000
Inference speed (Hz)	141 (one 64 s chunk per 7.1 ms)
Peak memory	101 MiB

The model is small and fast (Table II): one 64 s chunk takes 7.1 ms, and the full test set, 8.5 hours of IMU, is processed in under ten seconds. Training and inference both run in fp16 mixed precision; all reported results were produced this way. It is not causal, since attention is bidirectional over the chunk; an online deployment would either accept 64 s of latency or slide a shorter window, which we did not evaluate.

III. DEVELOPMENT EXPERIMENTS

We built our approach greedily: each component was added on top of the best configuration so far, trained once from scratch on `train`, and evaluated by its *local validation score*¹. `val` is held out from every model for which this score is reported, and the run-to-run spread of the score is about 0.005. The backbone without augmentation reaches 0.277; re-scaling gives 0.273, re-timing 0.202, the gravity head 0.193, the spectral branch 0.185 and the embodiment state vector 0.179. Re-timing is the largest single gain because it changes the frequency content and the ratio between kinematic and gravitational specific force, from which the model reads speed, whereas re-scaling only moves the target magnitude within a range the data already cover. The train-only twin of the submitted configuration (20k steps on `train`) scores 0.175; the ablation of the final configuration on the test set is in Section IV-D.

¹Throughout the report, *local validation score* denotes the challenge score computed with the organizers’ metric code on the released `val` split, as distinct from the numbers returned by the official scoring service on the test split.

Several directions did not yield noticeable gains on our local validation. Within our budget, they were stopped early once their validation curve fell behind the reference, so these observations are indicative rather than conclusive. Larger models did not help: a 22 M-parameter variant and dropout 0.2 were behind the reference at 8k steps (0.256 and 0.241 against 0.228), and a 128 s context trained to completion scored 0.256 against 0.200 at 64 s, losing mainly on the drone, whose flights are shorter than the chunk; the few hours per platform, not capacity, limit the model. Per-platform experts did not help: a drone-only model reached a local drone AVE of 0.404 m/s against 0.364 for the unified model, and fine-tuning the unified model on one platform for 1k steps changed that platform’s AVE by at most 0.002 m/s. Further augmentations, including left–right mirroring, yaw-isotropic rotation of level-mounted drone flights, a wider re-timing range, random mount rotation, sensor noise and bias perturbation, and trajectory mixing, did not yield gains.

IV. EXPERIMENTS

A. Official Challenge Performance

Table III summarizes the overall result. Our model scores 0.21367 on all 89 test sequences, a 60% reduction from the released TartanIMU baseline², and improves all three metrics: ATE₂₀ by 55%, AVE by 62% and RTE by 57%. All numbers in Tables III–V and VI were produced by the official scoring service³; the MD5 of every scored CSV is listed in the appendix. The baseline row is the released `unified.pt` scored by the same service. Since the test split carries no platform label, its platform-specific head was selected per trajectory with the platform predicted by the auxiliary classification head of our training run, a training-time head that is not part of the released checkpoint or of our inference path. The resulting AVE and ATE₂₀ coincide with the baseline numbers published by the organizers. Our model scores 0.28283 on the Kaggle public leaderboard, ninth at the close of the competition.

TABLE III
OVERALL CHALLENGE PERFORMANCE ON THE **FINAL (PRIVATE)**
LEADERBOARD. THE TARTANIMU SCORE IS

0.6 AVE/0.7356 + 0.4 ATE₂₀/3.1160; IT IS DIMENSIONLESS AND *lower is better* (THE ALL-ZERO SUBMISSION SCORES EXACTLY 1.000).

ATE₂₀ IS THE 20 M-SEGMENT TRAJECTORY ERROR AFTER SE(3) ALIGNMENT. **RTE DOES NOT ENTER THE RANKING** AND IS REPORTED FOR ANALYSIS ONLY; IT IS DEFINED IN THE CAPTION OF TABLE VI.

Method	Score ↓	ATE ₂₀ ↓	AVE ↓	RTE ↓
TartanIMU Baseline	0.53799	1.26056	0.46121	1.89552
Ours	0.21367	0.56268	0.17342	0.81807

B. Per-Platform Results

Table IV breaks the result down by platform. Our model reduces the error on every platform; the gain is largest on

²<https://huggingface.co/Tartan-IMU/TartanIMU>

³https://huggingface.co/spaces/Tartan-IMU/imu_odometry_challenge_scoring

TABLE IV
PERFORMANCE BY PLATFORM. RTE IS DEFINED IN THE CAPTION OF
TABLE VI.

Platform	ATE ₂₀ ↓	AVE ↓	RTE ↓
Wheeled / Car	0.38504	0.06799	0.30095
Handheld / Human	0.53125	0.05126	0.29518
Legged / Quadruped	0.39942	0.07339	0.36021
Aerial / Drone	0.93501	0.50103	2.31594
Average	0.56268	0.17342	0.81807

the car and the drone, whose AVE drops by 62% and 66% from the baseline (0.18001 and 1.45350 m/s), and smallest on the quadruped (34%, from 0.11056 m/s). The three ground platforms reach 0.05–0.07 m/s AVE; the full per-platform comparison with the baseline and the ablations is in Table VII of the appendix. The aerial platform remains an order of magnitude harder at 0.50103 m/s.

C. Per-Sequence Results

The error is concentrated in a few aerial sequences rather than evenly spread (Table VI; the statistics below are aggregated locally from the official per-sequence output). The ten sequences with the largest AVE are all aerial and carry 52% of the AVE summed over the 89 sequences; two of them (sequences 29 and 56, 17 s long) reach 3.6 and 3.9 m/s, while the median aerial sequence has 0.31 m/s. On the ground platforms the median AVE is 0.055 m/s and the maximum 0.28 m/s.

Fig. 2 shows individual validation trajectories of the train-only twin of the submitted model (the submitted model itself was trained on `val`), chosen per platform at the median and 75th percentile of local validation ATE₂₀ and integrated with the reference attitude. On the ground platforms the path follows the reference closely; on the drone it keeps the shape of the manoeuvre but misjudges its scale or drifts away within tens of seconds, far less than the baseline yet an order of magnitude above the ground platforms.

D. Ablation Studies

TABLE V
ABLATION OF THE THREE MAIN COMPONENTS. EACH ROW IS THE
SUBMITTED CONFIGURATION RETRAINED WITHOUT ONE COMPONENT
(`TRAIN + VAL`, 30K STEPS, ONE SEED) AND SCORED BY THE OFFICIAL
SERVICE ON ALL 89 TEST SEQUENCES.

Configuration	Score ↓	ATE ₂₀ ↓	AVE ↓	RTE ↓
Full model (submitted)	0.21367	0.56268	0.17342	0.81807
w/o embodiment state vector	0.21429	0.56936	0.17311	0.82351
w/o re-timing augmentation	0.27261	0.69879	0.22425	0.98269
w/o gravity head	0.21068	0.54495	0.17253	0.79243

Table V summarizes the effect of removing each of the three main components from the submitted configuration on the official scoring service. Re-timing is the component that matters: without it the score rises from 0.214 to 0.273, with a loss on every platform except the quadruped, largest on the

drone (AVE 0.50 to 0.68 m/s) and the car (ATE₂₀ 0.39 to 0.49 m). The other two components did not yield noticeable gains. Without the embodiment state vector, the score is 0.2143, and without the gravity head, 0.2107, differences of +0.0006 and −0.0030 are small and change sign across platforms (Table VII in the appendix). Both components improved the local validation score when they were added during development (Section III), but that gain does not carry over to the final configuration trained on `train` and `val` for 30k steps. We keep them in the submitted model because it was selected before this ablation was run, and we do not claim them as demonstrated gains.

E. Generalization and Failure Modes

We see no negative transfer between platforms: a drone-only model trained with the same budget was not better than the unified model on drone data (local validation AVE 0.404 vs. 0.364 m/s), so adding the other platforms costs nothing. The remaining error has two sources. To characterize them without analyzing the test data, we evaluate the `train`-only twin of the submitted model on the `val` trajectories.

First, and by far the larger, is the aerial platform, where the error follows the sensor mount rather than the motion. We measure the mount tilt of each trajectory as the mean angle between the sensor’s z axis and gravity, taken from the reference attitude $\mathbf{R}_t$ over the recording. Over the 48 aerial validation trajectories, flights with a tilt below 8° average 0.18 m/s AVE and flights above it average 0.39 m/s, whereas neither mean speed nor mean angular rate separates the trajectories as well. A tilted mount puts part of gravity into the horizontal axes, so the small tilt changes that make aerial velocity observable through the drag-tilt relation must be separated from a large, flight-specific offset, and a one-second window gives the model little to separate them with.

Second, and much smaller, is the integrated drift on slow ground trajectories: the handheld platform has the lowest testing AVE but the highest ATE₂₀ of the three, because at walking speed a 20 m segment spans about twenty windows and the same velocity error accumulates over a longer time.

V. INSIGHTS AND LESSONS LEARNED

A single inertial model across embodiments is feasible, and it needs less platform-specific machinery than we expected. A dedicated expert did not beat the shared model on its own platform, which agrees with the observation of TartanIMU [9] that training across platforms helps each of them, and the ablation shows that even the explicit embodiment conditioning is not necessary.

Data coverage, not model capacity, is the bottleneck of learned inertial odometry at this data scale. Larger models did not help, whereas augmentation that is exact under the physics of the sensor was the largest gain of this work. Augmentations common in prior work, such as yaw rotation and bias perturbation [15], leave the motion regime unchanged; re-timing in the form of (5) extends it, at the cost of nothing but a reference attitude, and we expect it to be useful for other IMU tasks that share this structure, such as pedestrian

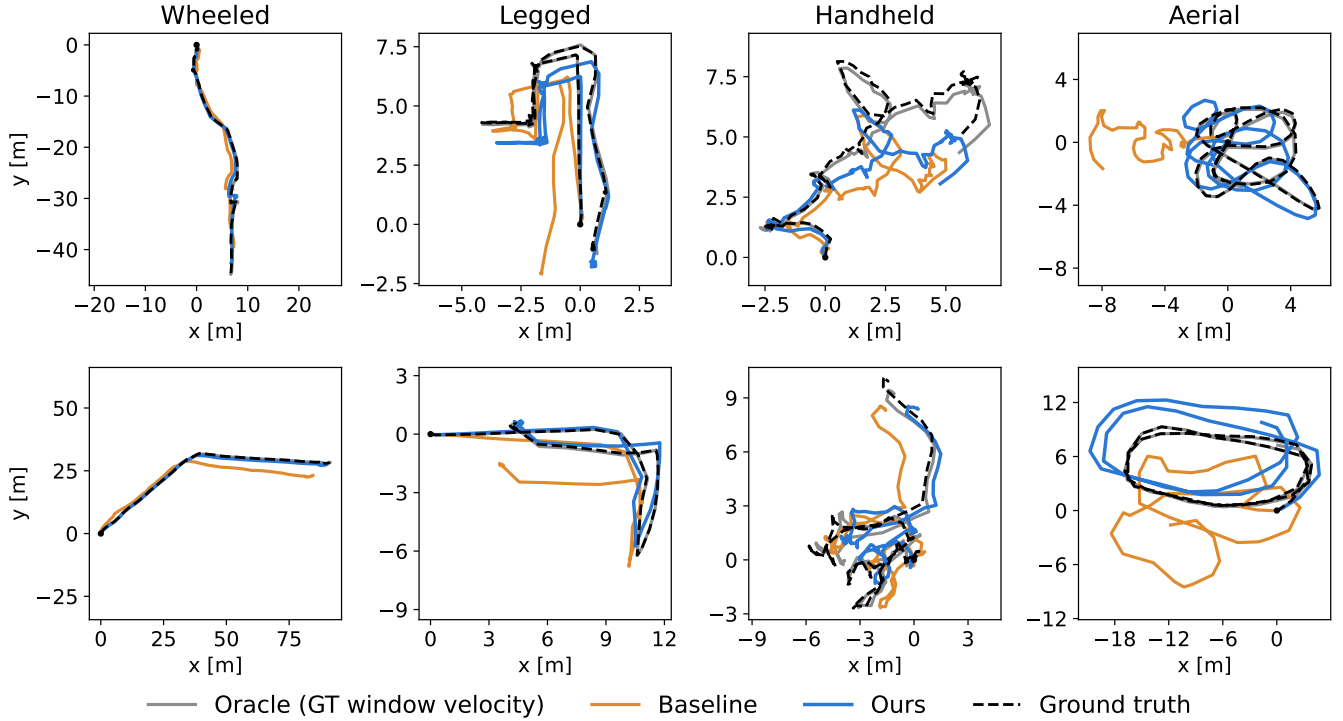


Fig. 2. Estimated trajectory on the validation split, from predicted body-frame velocity and rotated with the reference attitude, including a trajectory with median ATE_{20} (top) and 75th-percentile ATE_{20} (bottom) of each platform. Trajectories longer than 120 s are shown as the 120 s excerpt with the largest spatial extent. Locally computed ATE_{20} [m]/ AVE [m/s] of ours, with the baseline in parentheses, top and bottom row: wheeled 0.33/0.05 (0.72/0.13) and 0.40/0.09 (1.91/0.25); legged 0.25/0.10 (0.46/0.18) and 0.27/0.13 (0.31/0.19); handheld 0.51/0.08 (0.69/0.13) and 0.58/0.11 (0.66/0.15); aerial 0.55/0.19 (1.96/0.89) and 0.81/0.47 (2.09/1.36). “Ours” is the train-only twin of the submitted model (local validation score 0.175) without training on *val*; “Oracle” integrates the reference window-mean velocities and is the floor of the metric; “Baseline” is *unified.pt* with its platform-specific head selected by the reference platform label; following the organizers’ guidance, *val* should not be regarded as held out from it.

navigation [1], [4] and inertial motion capture [11], [12], [20].

Attitude is where the next gain lies. Feeding the reference gravity direction to the network improved the local validation score more than any architectural change we tried, which agrees with the finding of AirIO [10] that explicit attitude information improves body-frame velocity estimation, and with the tight coupling of network and attitude filter in TLIO [15]. Learning that attitude jointly, or estimating it with a complementary filter [32] or a preintegration-based estimator [7] that the network can trust, is in our view the most promising direction for a cross-platform model.

A. Feedback to the Organizers

The metric, the submission format and the per-sequence scoring service are clear. For a next edition, we would welcome a leave-one-embodiment-out track. Holding out one embodiment entirely would measure generalization to an unseen platform directly, and would reward methods that learn transferable physics over methods that memorize the motion prior of each platform. Additionally, a causal track with a bounded context window would separate what is achievable for online odometry.

VI. CONCLUSION

We presented a single model that predicts body-frame velocity for wheeled, legged, aerial, and handheld platforms from raw IMU with one set of weights, without platform labels, classification, or routing. It scores 0.21367 on the official scoring service, a 60% reduction from the released baseline. The model is trained from scratch on the released data only, and the ablation on the official test set shows that its accuracy comes from the kinematically consistent re-timing augmentation: removing it raises the score to 0.27261. The remaining error sits almost entirely in a small number of aerial sequences, which we identify as the natural target for future work.

ACKNOWLEDGMENT

We thank the organizers of the Tartan IMU Challenge for the dataset, the baseline and the per-sequence scoring service.

APPENDIX

A. Per-Sequence Results

Table VI lists ATE_{20} , AVE and RTE of every test sequence as returned by the official scoring service for the submitted CSV.

TABLE VI

PER-SEQUENCE CHALLENGE RESULTS OVER THE 89 TEST SEQUENCES. ATE, AVE AND RTE ARE PRODUCED BY THE OFFICIAL PER-SEQUENCE SCORING SERVICE WHOSE URL IS GIVEN ON THE FIRST PAGE. **RTE** (RELATIVE TRAJECTORY ERROR, M) FOLLOWS THE DEFINITION OF STURM *et al.* [33] AND IS THE METRIC USED IN AIRIO: OVER SLIDING WINDOWS OF A FIXED DURATION Δt , IT IS THE RMSE OF THE DIFFERENCE BETWEEN THE PREDICTED AND THE GROUND-TRUTH RELATIVE DISPLACEMENT, $RTE = \sqrt{\frac{1}{n} \sum_i \|(\hat{\mathbf{p}}_{i+\Delta t} - \hat{\mathbf{p}}_i) - (\mathbf{p}_{i+\Delta t} - \mathbf{p}_i)\|^2}$. UNLIKE ATE IT MEASURES *local* DRIFT AND IS INSENSITIVE TO ACCUMULATED GLOBAL OFFSET. THE WINDOW LENGTH IS FIXED TO $\Delta t = 5$ S FOR ALL PLATFORMS: AT $\Delta t = 1$ S THE METRIC IS NEARLY A RESTATEMENT OF AVE, WHILE THE SHORTEST AERIAL SEQUENCES ARE ONLY 19 S LONG, WHICH BOUNDS Δt FROM ABOVE.

Seq. ID	ATE ₂₀ ↓	AVE ↓	RTE ↓	Seq. ID	ATE ₂₀ ↓	AVE ↓	RTE ↓	Seq. ID	ATE ₂₀ ↓	AVE ↓	RTE ↓
1	0.85488	0.33772	1.84330	31	0.69516	0.31988	1.53064	61	0.83090	0.33797	1.18492
2	0.54408	0.34912	1.07975	32	0.26869	0.06752	0.21000	62	0.05869	0.01486	0.05346
3	0.70512	0.28009	1.13924	33	0.27293	0.03979	0.19161	63	0.57499	0.07341	0.37685
4	0.34874	0.17747	0.60644	34	0.96554	0.26750	1.22562	64	0.41916	0.05700	0.28770
5	0.65309	0.29929	1.37289	35	1.20157	0.37129	1.65506	65	0.16270	0.03986	0.16138
6	0.67118	0.08596	0.75643	36	0.95543	0.36931	1.81535	66	2.19315	0.23046	1.62298
7	0.69456	0.22610	0.96097	37	1.02741	1.09323	5.00530	67	0.11799	0.02939	0.13411
8	0.44200	0.03868	0.24424	38	1.03986	1.10575	4.70437	68	0.20233	0.02650	0.08801
9	0.60599	0.25754	0.88389	39	0.49038	0.19570	0.73096	69	0.89790	0.09033	0.53974
10	1.49084	0.13106	1.27872	40	0.84257	0.30450	1.85396	70	0.94864	0.09062	0.46380
11	0.96307	0.58191	1.73352	41	1.08193	0.37940	2.26790	71	0.16403	0.05763	0.16698
12	0.25768	0.15061	0.54632	42	0.40489	0.06849	0.29358	72	0.96524	0.50141	1.79542
13	0.21223	0.03903	0.17905	43	1.12073	0.27684	1.27671	73	0.41113	0.09403	0.39258
14	0.59391	0.38336	1.11345	44	0.27012	0.05156	0.31873	74	0.48768	0.24814	0.82306
15	0.78951	0.05557	0.33236	45	0.12098	0.04855	0.14051	75	0.62440	0.40934	1.22642
16	0.13328	0.03156	0.20450	46	0.22386	0.04250	0.16970	76	0.21945	0.07566	0.20705
17	1.44195	0.67331	2.86202	47	1.33181	0.46499	1.61747	77	1.11859	0.50979	2.09776
18	1.83151	0.65053	3.12861	48	0.21602	0.01556	0.11890	78	0.09647	0.02006	0.06490
19	0.49471	0.25072	1.00705	49	0.71093	0.22622	1.20182	79	0.25560	0.10087	0.46396
20	0.13687	0.04666	0.33852	50	0.24985	0.07970	0.33277	80	0.47397	0.19888	0.92269
21	0.66417	0.18834	1.04757	51	0.06971	0.02543	0.07300	81	0.79227	0.15437	0.52212
22	0.56399	0.39110	1.12253	52	0.25405	0.08913	0.52470	82	0.90080	0.18898	1.10380
23	1.02081	0.28109	1.97900	53	0.40582	0.28175	0.53390	83	0.46477	0.07177	0.38501
24	0.60897	0.04104	0.24908	54	0.22631	0.04296	0.19138	84	0.54393	0.05628	0.34875
25	0.10308	0.02531	0.08100	55	0.44693	0.18658	0.79325	85	0.13384	0.03239	0.11955
26	1.14793	0.11855	0.47434	56	4.19980	3.90469	19.74697	86	0.37663	0.02781	0.17512
27	0.44342	0.05695	0.27909	57	0.93074	0.37691	2.08094	87	0.32022	0.09882	0.45460
28	1.03765	0.27428	1.44227	58	0.25740	0.05446	0.27038	88	1.04901	0.33467	2.15334
29	4.14208	3.62997	19.35272	59	0.41568	0.08274	0.29055	89	0.07560	0.01843	0.05660
30	0.34028	0.31237	0.93051	60	0.19448	0.06897	0.22128				

B. Per-Platform Comparison

Table VII breaks Tables III and V down by platform and metric. Removing re-timing hurts every platform except the quadroped, most on the drone and the car; removing the embodiment state vector or the gravity head changes each platform by a few thousandths in either direction.

C. Reproducibility and Submission Artifacts

Kaggle team name. Haozhe Zhou.

Submitted CSV. `final_final_G.tv.csv`, 30644 rows plus header; all numbers in Tables III–IV and VI come from this file.

`md5 b6d8a00027780279b046534ea8ca20f1`

Other officially scored CSVs.

Released baseline, reference routing:

`596e0eea2c1cfa493db696a661d19ec5`

Ablation w/o embodiment state vector:

`3ba2cce182537ed0a28f2b4ade935e47`

Ablation w/o re-timing:

`8feca8e4ef2c48f82dc60434d3133c51`

Ablation w/o gravity head:

`960078f19f36d118863eaf1a40f8b25b`

Hugging Face repository.

Public commit `f8405c04` of 2026-09-20 (files uploaded in commit `4c311c8f`). It contains `model.pt`, `model.py`, `infer.py`, `config.json`, `requirements.txt`, `requirements-lock.txt` and the submitted `submission.csv`.

https://huggingface.co/haozheee/imu_odometry_challenge_iros26

Model checkpoint md5:

`e2d32e4b88b7db4bd05eee4b77bd2b6e`

Environment. Python 3.10.13, PyTorch 2.9.0+cu128, NumPy 1.23.5, pandas 2.3.0, one Quadro RTX 8000; pinned in `requirements-lock.txt`.

Reproduction check. In a fresh virtual environment built from the lock file, `infer.py` reproduces the submitted CSV byte for byte; with current package versions (PyTorch 2.14, NumPy 2.2) the predictions differ by 1.5×10^{-5} m/s on average and 1.8×10^{-3} m/s at most on our machine.

Seeds and variance. The reported score is a single training run (seed 0). The run-to-run spread measured by repeating one configuration is about ± 0.005 on the local validation score.

TABLE VII

PER-PLATFORM COMPARISON OF THE RELEASED BASELINE, THE SUBMITTED MODEL AND THE THREE ABLATION CONFIGURATIONS ON THE 89 TEST SEQUENCES. ALL NUMBERS ARE AGGREGATED FROM THE PER-SEQUENCE OUTPUT OF THE OFFICIAL SCORING SERVICE; THE LAST GROUP IS THE MACRO AVERAGE OVER THE FOUR PLATFORMS, AND THE BEST VALUE IN EACH COLUMN IS IN BOLD.

Configuration	Wheeled / Car			Handheld / Human			Legged / Quadruped			Aerial / Drone			Average		
	ATE ₂₀	AVE	RTE	ATE ₂₀	AVE	RTE	ATE ₂₀	AVE	RTE	ATE ₂₀	AVE	RTE	ATE ₂₀	AVE	RTE
TartanIMU baseline	1.174	0.180	1.001	1.023	0.101	0.468	0.640	0.111	0.481	2.205	1.454	5.632	1.261	0.461	1.896
Full model (submitted)	0.385	0.068	0.301	0.531	0.051	0.295	0.399	0.073	0.360	0.935	0.501	2.316	0.563	0.173	0.818
w/o embodiment state vector	0.347	0.064	0.270	0.568	0.053	0.305	0.370	0.072	0.348	0.993	0.504	2.371	0.569	0.173	0.824
w/o re-timing augmentation	0.494	0.081	0.391	0.688	0.061	0.343	0.367	0.074	0.356	1.246	0.681	2.841	0.699	0.224	0.983
w/o gravity head	0.331	0.061	0.251	0.519	0.053	0.290	0.385	0.071	0.340	0.945	0.505	2.289	0.545	0.173	0.792

External resources and held-out data. None. No model is initialized from the released TartanIMU checkpoint or any other pretrained weights. We only used the official released dataset. We used the challenge `val` as held out from every model whose local validation number is reported. The officially tested models used combined `train` and `val` for training, and tested with the organizers’ metric code on the official held-out tests from the official scoring service.

Compliance declaration.

- 1) **Single model with one shared set of weights?** Yes. One model with one shared set of weights for all four platforms.
- 2) **Platform classification, routing or specialization at inference?** No. There is no platform label, classifier, router or per-platform expert in the inference path. The network computes a continuous embodiment state vector from the IMU input and uses it to modulate its own normalization layers; the platform label is used during training only, as an auxiliary cross-entropy loss on that vector, and the classification heads are not in the released weights.
- 3) **Ensembling, checkpoint averaging or test-time augmentation?** None. Long trajectories are processed in overlapping 64s chunks blended with a squared-triangular window, which is chunked inference of one model.
- 4) **Test-set leakage, including model and checkpoint selection?** The test IMU was never used for training, pseudo-labelling or adaptation. All architecture and hyper-parameter decisions were made on the released `val` split. Checkpoints are the last step of a fixed budget, so no test information enters early stopping. The only use of test information is the final choice among three candidate configurations, made with the official scoring service; their scores are 0.21367 (submitted, Table III), 0.21503 and 0.21709, within run-to-run spread. The ablations of Table V were scored after the submission deadline and did not influence the submission.
- 5) **Development-time vs submitted system?** Same code and configuration. The submitted model is trained on `train+val` for 30k steps instead of `train` for 20k steps, and the training-only heads (platform, gravity) are stripped from the released checkpoint.

D. Final Predictions

The submitted predictions can be downloaded directly from the repository. The file is byte-identical to the one submitted to Kaggle and scored by the official service: 30 644 rows plus header, one row per `window_id` of `index/test_windows.csv`.

https://huggingface.co/haozheee/imu_odometry_challenge_iros26/resolve/main/submission.csv
`md5 b6d8a00027780279b046534ea8ca20f1 vx, vy, vz` is the predicted mean **body-frame** velocity of the window in m/s. No post-processing is applied after the network output, and there are no missing windows.

E. Model and Inference Code

The checkpoint, model definition, inference script, configuration and pinned environment are published at https://huggingface.co/haozheee/imu_odometry_challenge_iros26. Reproducing the submitted predictions takes two commands:

```
pip install -r requirements-lock.txt
python infer.py --test_dir <data>/test \
  --windows <data>/index/test_windows.csv \
  --out submission.csv
```

The script depends only on PyTorch, NumPy and pandas and needs no network access.

REFERENCES

- [1] E. Foxlin, "Pedestrian tracking with shoe-mounted inertial sensors," *IEEE Computer Graphics and Applications*, vol. 25, no. 6, pp. 38–46, Nov. 2005.
- [2] R. Harle, "A Survey of Indoor Inertial Positioning Systems for Pedestrians," *IEEE Communications Surveys & Tutorials*, vol. 15, no. 3, pp. 1281–1293, 2013.
- [3] H. Yan, Q. Shan, and Y. Furukawa, "RIDI: Robust IMU Double Integration," in *Computer Vision – ECCV 2018*, V. Ferrari, M. Hebert, C. Sminchisescu, and Y. Weiss, Eds. Cham: Springer International Publishing, 2018, pp. 641–656.
- [4] S. Herath, H. Yan, and Y. Furukawa, "RoNIN: Robust Neural Inertial Navigation in the Wild: Benchmark, Evaluations, & New Methods," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, May 2020, pp. 3146–3152.
- [5] P. D. Groves, *Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems*, 2nd ed. Artech House, 2013.
- [6] M. Brossard, A. Barrau, and S. Bonnabel, "AI-IMU Dead-Reckoning," *IEEE Transactions on Intelligent Vehicles*, vol. 5, no. 4, pp. 585–595, Dec. 2020.
- [7] C. Forster, L. Carlone, F. Dellaert, and D. Scaramuzza, "On-Manifold Preintegration for Real-Time Visual-Inertial Odometry," *IEEE Transactions on Robotics*, vol. 33, no. 1, pp. 1–21, Feb. 2017.
- [8] R. Buchanan, V. Agrawal, M. Camurri, F. Dellaert, and M. Fallon, "Deep IMU Bias Inference for Robust Visual-Inertial Odometry With Factor Graphs," *IEEE Robotics and Automation Letters*, vol. 8, no. 1, pp. 41–48, Jan. 2023.
- [9] S. Zhao, S. Zhou, R. Blanchard, Y. Qiu, W. Wang, and S. Scherer, "Tartan IMU: A Light Foundation Model for Inertial Positioning in Robotics," in *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2025, pp. 22 520–22 529.
- [10] Y. Qiu, C. Xu, Y. Chen, S. Zhao, J. Geng, and S. Scherer, "AirIO: Learning Inertial Odometry With Enhanced IMU Feature Observability," *IEEE Robotics and Automation Letters*, vol. 10, no. 9, pp. 9368–9375, Sep. 2025.
- [11] H. Zhou, R. Arakawa, Y. Agarwal, and M. Goel, "IMUCoCo: Enabling Flexible On-Body IMU Placement for Human Pose Estimation and Activity Recognition," in *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology*, ser. UIST '25. New York, NY, USA: Association for Computing Machinery, Sep. 2025, pp. 1–16.
- [12] R. Hori, J.-T. Song, Z. Luo, J. Cao, S. Shin, H. Saito, and K. Kitani, "Ground Reaction Inertial Poser: Physics-based Human Motion Capture from Sparse IMUs and Insole Pressure Sensors," 2026, pp. 28 435–28 445.
- [13] D. Titterton and J. L. Weston, *Strapdown Inertial Navigation Technology*. IET, 2004.
- [14] C. Chen, X. Lu, A. Markham, and N. Trigoni, "IONet: Learning to Cure the Curse of Drift in Inertial Odometry," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, Apr. 2018.
- [15] W. Liu, D. Caruso, E. Ilg, J. Dong, A. I. Mourikis, K. Daniilidis, V. Kumar, and J. Engel, "TLIO: Tight Learned Inertial Odometry," *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 5653–5660, Oct. 2020.
- [16] S. Sun, D. Melamed, and K. Kitani, "IDOL: Inertial Deep Orientation-Estimation and Localization," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 7, pp. 6128–6137, May 2021.
- [17] X. Cao, C. Zhou, D. Zeng, and Y. Wang, "RIO: Rotation-equivariance supervised learning of robust inertial odometry," in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2022, pp. 6604–6613.
- [18] M. Brossard, S. Bonnabel, and A. Barrau, "Denoising IMU Gyroscopes With Deep Learning for Open-Loop Attitude Estimation," *IEEE Robotics and Automation Letters*, vol. 5, no. 3, pp. 4796–4803, Jul. 2020.
- [19] M. Faessler, A. Franchi, and D. Scaramuzza, "Differential Flatness of Quadrotor Dynamics Subject to Rotor Drag for Accurate Tracking of High-Speed Trajectories," *IEEE Robotics and Automation Letters*, vol. 3, no. 2, pp. 620–626, Apr. 2018.
- [20] X. Yi, Y. Zhou, and F. Xu, "TransPose: real-time 3D human translation and pose estimation with six inertial sensors," *ACM Trans. Graph.*, vol. 40, no. 4, pp. 86:1–86:13, Jul. 2021.
- [21] E. Perez, F. Strub, H. d. Vries, V. Dumoulin, and A. Courville, "FiLM: Visual Reasoning with a General Conditioning Layer," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, Apr. 2018.
- [22] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, u. Kaiser, and I. Polosukhin, "Attention is All you Need," in *Advances in Neural Information Processing Systems*, vol. 30. Curran Associates, Inc., 2017.
- [23] W. Peebles and S. Xie, "Scalable diffusion models with transformers," in *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, 2023, pp. 4172–4182.
- [24] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer Normalization," Jul. 2016.
- [25] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "RoFormer: Enhanced transformer with Rotary Position Embedding," *Neurocomputing*, vol. 568, p. 127063, Feb. 2024.
- [26] A. Gulati, J. Qin, C.-C. Chiu, N. Parmar, Y. Zhang, J. Yu, W. Han, S. Wang, Z. Zhang, Y. Wu, and R. Pang, "Conformer: Convolution-augmented Transformer for Speech Recognition," May 2020.
- [27] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-End Object Detection with Transformers," in *Computer Vision – ECCV 2020*, A. Vedaldi, H. Bischof, T. Brox, and J.-M. Frahm, Eds. Cham: Springer International Publishing, 2020, pp. 213–229.
- [28] T. T. Um, F. M. Pfister, D. Pichler, S. Endo, M. Lang, S. Hirche, U. Fietzek, and D. Kulić, "Data augmentation of wearable sensor data for parkinson's disease monitoring using convolutional neural networks," in *Proceedings of the 19th ACM international conference on multimodal interaction*, 2017, pp. 216–220.
- [29] B. K. Iwana and S. Uchida, "An empirical survey of data augmentation for time series classification with neural networks," *Plos one*, vol. 16, no. 7, p. e0254841, 2021.
- [30] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," Jan. 2019.
- [31] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu, "Mixed Precision Training," Feb. 2018.
- [32] S. O. H. Madgwick, A. J. L. Harrison, and R. Vaidyanathan, "Estimation of IMU and MARG orientation using a gradient descent algorithm," in *2011 IEEE International Conference on Rehabilitation Robotics*, Jun. 2011, pp. 1–7.
- [33] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A benchmark for the evaluation of RGB-D SLAM systems," in *Proc. IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*, 2012.